

实验作业 5

实验作业 5

一、实验任务

任务一至三

任务四

1. 获取文档向量 (Document Embeddings)
2. 计算文档向量的哈希值 (hash) & 创建 hash tables
3. 实现 Approximate K-NN

二、作业提交

三、参考文献及资料

本次实验旨在帮助同学初步掌握 NLP 中的基础方法和典型技术路线。通过完成一系列循序渐进的实验任务，学生将学习从传统机器学习的文本分类方法，到词向量的分布式表示，再到基于局部敏感哈希 (Locality-Sensitive Hashing, LSH) 的高效文本检索与简单机器翻译模型的实现。

一、实验任务

本次实验包含 4 个任务，分数占比如下：

- 任务一：Logistic Regression for Sentiment Analysis (20%)
- 任务二：Naive Bayes for Sentiment Analysis (20%)
- 任务三：Word Vectors (20%)
- 任务四：Locality-Sensitive Hashing (40%)

上述任务在知识点上循序渐进，建议按照先后顺序依次完成。

任务一至三

在任务一、二、三中，已为同学们提供相应的 Jupyter Notebook 文件，对应关系如下：

- 任务一： `Task1/Logistic-Regression.ipynb`
- 任务二： `Task2/Naive-Bayes.ipynb`
- 任务三： `Task3/Word-Vectors.ipynb`

在任务一和任务二中，我们首先使用两种最基础、经典的机器学习方法 (Logistic Regression & Naive Bayes) 来完成 Tweets 的情感分析 (Sentiment Analysis，也是最典型的文本分类任务之一)。

在**任务三**中，我们将通过一些案例来学习词向量（Word Vectors）的原理和性质，并学习常见的词向量降维算法 PCA（Principle Component Analysis）。

要求代码能正常运行且输出符合预期。

任务四

本任务旨在让同学掌握局部敏感哈希（Locality-Sensitive Hashing, LSH）的基本原理及其在文本相似度搜索中的应用。通过本任务，你将能够将文本表示为向量、构建哈希表，并实现高效的近似 K 近邻搜索，及查找与某个 tweet 最相似的 tweet。

在 **Task4** 目录下给出了 `utils.py`，里面有你在本任务中可以使用的辅助函数。

在开始实验时，需要先引入必要的依赖并做一些准备工作：

```
1 import pickle
2 import nltk
3 import numpy as np
4 from nltk.corpus import twitter_samples
5
6 # download stopwords and twitter samples from nltk
7 nltk.download('stopwords')
8 nltk.download('twitter_samples')
9
10 # use the subset of word embeddings
11 en_embeddings_subset = pickle.load(open("en_embeddings.p", "rb"))
12 fr_embeddings_subset = pickle.load(open("fr_embeddings.p", "rb"))
13
14 # get the positive and negative tweets
15 all_positive_tweets = twitter_samples.strings('positive_tweets.json')
16 all_negative_tweets = twitter_samples.strings('negative_tweets.json')
17 all_tweets = all_positive_tweets + all_negative_tweets
```

在本任务的实现中，除了上面导入的包和 `utils.py` 中给出的函数，不要导入其它工具。

下面给出大致的实现思路：

1. 获取文档向量（Document Embeddings）

- 实现函数 `get_document_embedding()`

```
1 def get_document_embedding(tweet, en_embeddings):
2     '''
3     Input:
4         - tweet: a string
5         - en_embeddings: a dictionary of word embeddings
6     Output:
7         - doc_embeddings: sum of all word embeddings in the tweet
8     '''
9     ...
```

测试脚本与输出示例：

```
1 custom_tweet = "RT @Twitter @chapagain Hello There! Have a great day. :) #good #morning  
http://chapagain.com.np"  
2 tweet_embedding = get_document_embedding(custom_tweet, en_embeddings_subset)  
3 tweet_embedding[-5:]
```

```
1 # Expected Output  
2 array([-0.00268555, -0.15378189, -0.55761719, -0.07216644, -0.32263184])
```

- **说明：**此处所谓 Document Embedding 指的是某个 tweet 的向量表示。此处我们采用最简单的 BOW (Bag-of-words) document model，即忽略 tweet 中出现单词的先后顺序，因此你只需要将 tweet 中所有单词的词向量累加起来即可。

- 进一步实现函数 `get_document_vecs()`

```
1 def get_document_vecs(all_docs, en_embeddings):  
2     '''  
3     Input:  
4         - all_docs: list of strings - all tweets in our dataset.  
5         - en_embeddings: dictionary with words as the keys and their embeddings as the values.  
6     Output:  
7         - document_vec_matrix: matrix of tweet embeddings.  
8         - ind2Doc_dict: dictionary with indices of tweets in vecs as keys and their embeddings  
9         as the values.  
10    '''  
11    ...
```

测试脚本与输出示例：

```
1 document_vecs, ind2Tweet = get_document_vecs(all_tweets, en_embeddings_subset)  
2 print(f"length of dictionary {len(ind2Tweet)}")  
3 print(f"shape of document_vecs {document_vecs.shape}")
```

```
1 # Expected Output  
2 length of dictionary 10000  
3 shape of document_vecs (10000, 300)
```

2. 计算文档向量的哈希值 (hash) & 创建 hash tables

局部敏感哈希 (LSH) 的思想大致可以概括为将向量空间划分为多个部分，然后再分别在不同的部分中找到最相似的向量。

如何划分向量空间？不难想到，对于一个平面，一条直线可以将其划分为两部分；而对于一个三维空间，一个平面又能将其划分为两部分。我们将这种将一个向量空间划分为两部分的对象称为 **plane**（直线是平面的 plane，平面是三维空间的 plane，以此类推，显然 plane 总是比它划分的向量空间低一维）。

在这里，我们使用**法向量 (normal vector)** 来表示一个 plane，即位于 plane 内的任意一个向量与法向量的点积都是 0。根据一个向量和法向量的点积为正或负，我们可以判断它位于这个 plane 的哪一侧（相信你应该不难理解，我们在这里说的 plane 其实是一个**过原点的 plane**）。由此：

1. 如果定义 N 个 plane (N 个法向量)，那么一个向量在空间中位于哪一区域可以用一个长度为 N 的数组表示，每个位置对应一个 plane，值为 1 或者 0，分别表示位于 plane 的哪一侧（点积正负）。
2. 对于一个向量，如果计算出了上面的数组，长度为 N ， h_i 表示数组索引为 i 的值，那么向量的哈希值为 $hash = \sum_{i=0}^{N-1} (2^i \times h_i)$ ，桶的数量是 2^N （想想为什么？）。

顺便一说，你可能会发现在某些情况下， N 个过原点 plane 可能无法将整个向量空间划分为 2^N 个区域（最简单的例子是平面上过原点的 3 条直线只能分出 6 个区域），但是实际中为了方便，我们显然没必要考虑这些情况，有些“不存在”的桶就让它空着就行。

了解了以上理论知识，就开始动手实现吧：

- 首先帮你定义了下面的变量：

```
1 N_VECS = len(all_tweets)    # How many vectors.
2 N_DIMS = len(ind2tweet[1])  # Vector dimensionality.
3
4 # The number of planes.
5 N_PLANES = 10
6 # Number of times to repeat the hashing to improve the search.
7 N_UNIVERSES = 25
8
9 np.random.seed(0)
10 planes_l = [np.random.normal(size=(N_DIMS, N_PLANES))
11              for _ in range(N_UNIVERSES)]
```

- **说明：** `planes_l` 的每个元素的每一列代表什么？

- 实现函数 `hash_value_of_vector()`：

```
1 def hash_value_of_vector(v, planes):
2     """Create a hash for a vector; hash_id says which random hash to use.
3     Input:
4         - v: vector of tweet. It's dimension is (1, N_DIMS)
5         - planes: matrix of dimension (N_DIMS, N_PLANES) - the set of planes that divide up the
        region
6     Output:
7         - res: a number which is used as a hash for your vector
8
9     """
10     ...
```

测试脚本与输出示例：


```

1 np.random.seed(0)
2 idx = 0
3 planes = planes_l[idx] # get one 'universe' of planes to test the function
4 vec = np.random.rand(1, 300)
5 print(f"The hash value for this vector,",
6       f"and the set of planes at index {idx},",
7       f"is {hash_value_of_vector(vec, planes)}")

```

```

1 # Expected Output
2 The hash value for this vector, and the set of planes at index 0, is 768

```

- 接着实现函数 `make_hash_table()` 来创建哈希表：

```

1 def make_hash_table(vecs, planes):
2     """
3     Input:
4         - vecs: list of vectors to be hashed.
5         - planes: the matrix of planes in a single "universe", with shape (embedding
6           dimensions, number of planes).
7     Output:
8         - hash_table: dictionary - keys are hashes, values are lists of vectors (hash buckets)
9         - id_table: dictionary - keys are hashes, values are list of vectors id's
10           (it's used to know which tweet corresponds to the hashed vector)
11     """
12     ...

```

测试脚本与输出示例：

```

1 np.random.seed(0)
2 planes = planes_l[0] # get one 'universe' of planes to test the function
3 vec = np.random.rand(1, 300)
4 tmp_hash_table, tmp_id_table = make_hash_table(document_vecs, planes)
5
6 print(f"The hash table at key 0 has {len(tmp_hash_table[0])} document vectors")
7 print(f"The id table at key 0 has {len(tmp_id_table[0])}")
8 print(f"The first 5 document indices stored at key 0 of are {tmp_id_table[0][0:5]}")

```

```

1 # Expected Output
2 The hash table at key 0 has 3 document vectors
3 The id table at key 0 has 3
4 The first 5 document indices stored at key 0 of are [3276, 3281, 3282]

```

之后就可以创建所有的 hash tables 了：

```

1 hash_tables = []
2 id_tables = []
3 for universe_id in range(N_UNIVERSES): # there are 25 hashes
4     print('working on hash universe #:', universe_id)
5     planes = planes_l[universe_id]
6     hash_table, id_table = make_hash_table(document_vecs, planes)
7     hash_tables.append(hash_table)
8     id_tables.append(id_table)

```

3. 实现 Approximate K-NN

最后，我们将使用 LSH 来实现近似 k 邻近（Approximate K-NN）算法，来查找和给出的 tweet 相似的 tweets。

你的任务就是实现 `approximate_knn()` 这一函数：

```
1 def approximate_knn(doc_id, v, planes_l, k=1, num_universes_to_use=N_UNIVERSES):
2     """Search for k-NN using hashes."""
3     assert num_universes_to_use <= N_UNIVERSES
4     ...
```

输入参数说明：

- `doc_id`： `all_tweets` 中的索引位置。
- `v`： 对应 `all_tweets[doc_id]` 的文档向量。
- `planes_l`： plane 列表，即之前创建的全局变量。
- `k`： 要检索的最近邻数量。
- `num_universes_to_use`： 为了节省时间，我们可以只使用部分哈希空间（universe），而不必使用全部。默认值为 `N_UNIVERSES`，在本次作业中为 25。

`approximate_knn()` 会从所有向量中筛选出与输入向量 `v` 落入同一个桶的候选向量集合，然后只在这一小部分候选向量上执行常规的 k 最近邻搜索，而不是在所有 tweets 上进行查询。

要求使用余弦相似度（Cosine Similarity）来比较两个向量。

下面是一个使用 Approximate K-NN 例子：

```
1 # Sample
2 doc_id = 0
3 doc_to_search = all_tweets[doc_id]
4 vec_to_search = document_vecs[doc_id]
5
6 nearest_neighbor_ids = approximate_knn(
7     doc_id,
8     vec_to_search,
9     planes_l,
10    k=3,
11    num_universe_to_use=5
12 )
13
14 print(f"Nearest neighbors for document {doc_id}")
15 print(f"Document contents: {doc_to_search}")
16 print("")
17
18 for neighbor_id in nearest_neighbor_ids:
19     print(f"Nearest neighbor at document id {neighbor_id}")
20     print(f"document contents: {all_tweets[neighbor_id]}")
```

输出示例：

```
1 Nearest neighbors for document 0
2 Document contents: #FollowFriday @France_Inte @PKuchly57 @Milipol_Paris for being top engaged members
  in my community this week :)
3
4 Nearest neighbor at document id 2140
5 document contents: @PopsRamjet come one, every now and then is not so bad :)
6 Nearest neighbor at document id 701
7 document contents: With the top cutie of Bohol :) https://t.co/Jh7F6U46UB
8 Nearest neighbor at document id 51
9 document contents: #FollowFriday @France_Espana @reglisse_menthe @CCI_inter for being top engaged
  members in my community this week :)
```

至此，本次实验全部结束！🎉

二、作业提交

本次作业采取**个人提交**的方式，请各位同学务必自主独立完成，一旦发现学术不端行为，本次实验成绩作废。

提交格式：

- 保留作业文件夹原有目录结构，在此基础上：
 - 补全**任务一、二、三**中要求的代码实现，保证输出符合预期。
 - **任务四**的代码放在 `Task4` 目录中，可以提供简要的 README 来对代码进行说明。
- 将以上内容打成压缩包提交，压缩包命名为“**学号-姓名-实验作业5**”。

作业截止提交日期：12 月 29 日。

三、参考文献及资料

[1] [nltk twitter_samples dataset](#)

[2] [Additive smoothing - Wikipedia](#)

[3] [Google Code Archive - Long-term storage for Google Code Project Hosting.](#)

[4] [Principal component analysis - Wikipedia](#)